

AI-Generated Steganographic Malware: Evaluating Detection and Evasion in Modern Anti- Virus Systems

Master of Science in Computer Science at the University of
Johannesburg

Student: Thendo Siphuma

Supervisor: Prof Daniel Du Plessis

Table of Contents

Chapter 3: Research Methodology	1
3.1. Introduction.....	1
3.2. Chapter objectives	1
3.3. Chapter objectives	1
3.4. System requirements and threat model	1
3.3.1. Functional requirements (detection accuracy, throughput, integration)	1
3.3.2.	1
3.3.3.	2
3.4. Proposed system architecture (high-level)	2
3.4.1.	2
3.4.2.	2
3.4.3.	2
3.5. Data sources, dataset design and sampling strategy.....	2
3.5.1.	2
3.5.2.	2
3.5.3.	2
3.5.3.	3
3.5.3.	3
3.6 Feature engineering and extraction	3
Explains how meaningful features were extracted from images to support detection. 3	
3.6.1.	3
3.6.2.	3
3.6.3.	3
3.6.4.	3
3.7. YARA rule design and engineering (signature-based stage)	4
Explains how YARA rules were designed to detect known steganographic patterns. ...	4
3.7.1.	4
3.7.2.	4
3.7.3.	4
3.8. Machine learning anomaly detection (deep analysis stage)	4

Explains how machine learning was used to detect unknown or evasive steganographic malware.	4
3.8.1.	4
3.8.2.	5
3.8.3.	5
3.8.3.	5
3.9. Hybrid pipeline integration and decision fusion	5
Explains how YARA and machine learning outputs are combined.	5
3.9.1.	5
3.9.2.	5
3.9.3.	5
3.10. Experimental setup and tools	6
Details the technical environment used to conduct experiments.	6
3.10.1.....	6
3.10.2.....	6
3.10.3.....	6
3.10. Evaluation framework and metrics	6
Explains how the performance of the proposed system was evaluated.	6
Explain containment, sandboxing, and avoidance of live internet propagation.	6
3.10.2.....	6
3.10.3.....	6
3.10.3.....	6
3.10.3.....	7
3.11. Chapter Summary	7
REFERENCES	7

Chapter 3: Research Methodology

3.1. Introduction

Introduces the purpose of the chapter and explains that it describes how the research was conducted, including system design, data preparation, experiments, and evaluation methods.

3.2. Chapter objectives

3.2.1 Research Aims

3.2.1 Research Questions

3.3. Chapter objectives

3.3.1 Quantitative / experimental justification

Justifies the use of quantitative methods by explaining that system performance is measured using numerical metrics such as detection rate and false positives.

3.3.2 Mixed-methods components (technical analysis and qualitative insight)

Explains the inclusion of technical analysis and expert interpretation alongside numerical results to better understand detection behaviour.

3.4. System requirements and threat model

Defines what the proposed detection system must achieve and outlines the type of attacker and steganographic threats considered.

3.3.1. Functional requirements (detection accuracy, throughput, integration)

Specifies the core functions the system must perform, including detecting steganographic malware and integrating YARA with machine learning.

3.3.2. Non-functional requirements (performance, scalability, explainability)

Describes system qualities such as efficiency, scalability, and the ability to explain detection decisions.

3.3.3. Threat model and assumptions (adversary capabilities, cover media types)

Defines assumptions about attacker capabilities, steganography techniques, and the types of media used to hide malware.

3.4. Proposed system architecture (high-level)

Provides an overview of the architecture of the proposed hybrid detection system.

3.4.1. Two-step hybrid pipeline overview (YARA → machine learning deep analysis)

Explains the two-stage detection process where YARA performs initial filtering and machine learning performs deeper anomaly analysis.

3.4.2. Component responsibilities and data flow

Describes the role of each system component and how data moves through the detection pipeline.

3.4.3. Orchestration, logging, and feedback mechanisms

Explains how system components are coordinated, how results are logged, and how feedback improves detection.

3.5. Data sources, dataset design and sampling strategy

Explains how datasets were sourced, created, and structured for experiments.

3.5.1. Benign image collections (sources and selection criteria)

Describes where benign images were obtained and how they were selected.

3.5.2. Malware and steganographic sample sources (public repositories and generated samples)

Explains how malware samples and steganographic images were collected or generated.

3.5.3. AI-generated steganography vs LSB-based steganography generation procedures

Describes how different steganographic techniques were applied to generate malicious samples.

3.5.3. Labelling strategy, class balance, and dataset splits (training, validation, testing)

Explains how data was labelled, balanced, and split for machine learning experiments.

3.5.3. Data preprocessing and augmentation

Describes how different steganographic techniques were applied to generate malicious samples.

3.6 Feature engineering and extraction

Explains how meaningful features were extracted from images to support detection.

3.6.1. Pixel- and byte-level features (distributions and local statistics)

Describes low-level features derived from pixel and byte distributions.

3.6.2. Global features (entropy, file structure, metadata)

Explains the use of global statistical and structural features to identify anomalies.

3.6.3. Transform-domain features (DCT and frequency-based features)

Describes frequency-based features used to detect subtle steganographic changes.

3.6.4. Feature selection and dimensionality reduction techniques

Explains how irrelevant or redundant features were reduced to improve model performance.

3.7. YARA rule design and engineering (signature-based stage)

Explains how YARA rules were designed to detect known steganographic patterns.

3.7.1. Rule design principles and templates (strings, hexadecimal patterns, regular expressions, conditions)

Describes the structure and logic used to create effective YARA rules.

3.7.2. Targeted steganographic indicators for YARA (entropy thresholds and structural markers)

Explains the specific indicators used by YARA to flag potential steganographic malware.

3.7.3. Rule testing, optimisation, and false-positive reduction

Describes how rules were tested and refined to reduce incorrect detections.

3.8. Machine learning anomaly detection (deep analysis stage)

Explains how machine learning was used to detect unknown or evasive steganographic malware.

3.8.1. Rationale for unsupervised and anomaly-based learning

Justifies the use of anomaly detection to identify previously unseen malware.

3.8.2. Candidate models (autoencoders, isolation forest, one-class SVM, clustering techniques)

Introduces the machine learning models considered for anomaly detection.

3.8.3. Training, validation, and hyperparameter tuning process

Describes how models were trained, validated, and optimised.

3.8.3. Handling concept drift and adversarial evasion techniques

Explains how the system accounts for evolving malware behaviour.

3.9. Hybrid pipeline integration and decision fusion

Explains how YARA and machine learning outputs are combined.

3.9.1. Orchestration strategy: YARA-first filtering with machine learning escalation

Describes how files move from signature detection to deeper analysis.

3.9.2. Fusion logic, thresholds, and confidence scoring

Explains how final detection decisions are made.

3.9.3. Feedback loop for updating YARA rules based on machine learning findings

Describes how machine learning insights improve YARA rules over time.

3.10. Experimental setup and tools

Details the technical environment used to conduct experiments.

3.10.1. Software environment and tools

Lists the tools and frameworks used for detection and analysis.

3.10.2. Hardware environment, virtualisation, and safe containment

Explains how experiments were safely executed in isolated environments.

3.10.3. Reproducibility measures (version control, scripts, dataset management)

Describes steps taken to ensure experiments can be replicated.

3.10. Evaluation framework and metrics

Explains how the performance of the proposed system was evaluated.

Explain containment, sandboxing, and avoidance of live internet propagation.

3.10.2. Primary evaluation metrics

Defines metrics used to measure detection effectiveness.

3.10.3. Secondary performance metrics

Describes metrics related to efficiency and resource usage.

3.10.3. Experimental scenarios and baseline comparisons

Explains how the proposed system was compared to baseline approaches.

3.10.3. Statistical analysis and significance testing

Describes methods used to validate the reliability of results.

3.11. Chapter Summary

Summary of Methodological Steps

Concisely restate the chosen methods and rationale.

Transition to Next Chapter

Explain how this methodology leads into Chapter 4: System Design and Development.

REFERENCES